

ЛАБОРАТОРНА РОБОТА № 1

1.1 Тема

Основи роботи з Python: змінні, типи даних та операції

1.2 Мета роботи

Ознайомитися з основними поняттями мови програмування Python: змінними, базовими типами даних та арифметичними операціями над ними. Набути практичних навичок створення простих програм з використанням введення та виведення даних.

1.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

- знати основні типи даних у Python (int, float, str, bool);
- розуміти принципи оголошення та використання змінних;
- вміти застосовувати арифметичні оператори;
- вміти використовувати функції input() та print();
- скласти прості програми лінійної структури.

1.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

1.5 Теоретичні відомості

Поняття алгоритму та програми

Алгоритм – це а скінченна послідовність чітко визначених дій, виконання яких приводить до розв'язання поставленої задачі. У програмуванні алгоритм

реалізується у вигляді програми – тексту, написаного мовою програмування, який може бути виконаний комп'ютером.

Залежно від логіки виконання команд алгоритми поділяються на:

- 1) лінійні;
- 2) розгалужені;
- 3) циклічні.

У межах даної лабораторної роботи розглядаються алгоритми лінійної структури.

Лінійна структура програми

Лінійна структура – це найпростіший тип алгоритму, у якому всі команди виконуються послідовно, одна за одною, у тому порядку, в якому вони записані в програмі. У таких програмах відсутні розгалуження та цикли, а виконання не залежить від умов.

Лінійні програми застосовуються для:

- обчислення математичних виразів;
- перетворення вхідних даних;
- виконання простих розрахунків;
- демонстрації базових можливостей мови програмування.

Структура простої лінійної програми на Python

Типова програма лінійної структури складається з таких етапів:

- 1) Введення початкових даних.
- 2) Обробка даних (обчислення).
- 3) Виведення результатів.

У Python ці етапи реалізуються за допомогою стандартних конструкцій та функцій.

Принципи оголошення та використання змінних у мові програмування Python

Поняття змінної

Змінна в мові програмування Python – це іменований об'єкт, який використовується для зберігання даних під час виконання програми. Кожна змінна має:

- ім'я (ідентифікатор);
- значення;
- тип даних, який визначається автоматично під час присвоєння значення.

На відміну від багатьох мов програмування, у Python не потрібно попередньо оголошувати тип змінної. Тип змінної визначається динамічно на основі значення, яке їй присвоюється.

Оголошення (створення) змінних

Змінна створюється в момент першого присвоєння значення за допомогою оператора `=`.

Загальний вигляд:

```
ім'я_змінної = значення
```

Приклади:

```
a = 10
b = 3.14
name = "Python"
is_active = True
```

У наведених прикладах Python автоматично визначає тип змінної:

- `a` – ціле число (`int`);
- `b` – число з плаваючою комою (`float`);
- `name` – рядок (`str`);
- `is_active` – логічний тип (`bool`).

Правила іменування змінних

Імена змінних у Python повинні відповідати таким правилам:

- Ім'я може складатися з літер латинського алфавіту, цифр та символу підкреслення `_`.

- Ім'я не може починатися з цифри.
- Ім'я змінної чутливе до регістру (value і Value – різні змінні).
- Заборонено використовувати ключові слова Python (if, for, while, print тощо).

Правильно:

```
count = 5
user_name = "Ivan"
total_sum = 100
```

Неправильно:

```
2value = 10
user-name = "Ivan"
for = 7
```

Рекомендується використовувати змістовні імена, які відображають призначення змінної.

Основні типи даних у мові програмування Python

Однією з ключових особливостей мови програмування Python є динамічна типізація, тобто тип змінної визначається автоматично залежно від значення, яке їй присвоюється. На початковому етапі вивчення мови програмування особливу увагу приділяють базовим типам даних, оскільки вони є основою для створення більш складних програм.

До основних типів даних у Python належать: цілі числа (int), дійсні числа (float), рядки (str) та логічні значення (bool).

Цілий тип даних (int)

Тип даних int використовується для зберігання цілих чисел без дробової частини. У Python цілі числа можуть мати довільну довжину, обмежену лише обсягом пам'яті комп'ютера.

Приклади цілих чисел:

- додатні числа: 5, 12, 100
- від'ємні числа: -3, -25
- нуль: 0

Приклад створення змінної типу int:

```
a = 10  
b = -7
```

Цілий тип даних широко використовується для:

- підрахунку кількості об'єктів;
- нумерації;
- виконання арифметичних операцій без дробової частини;
- роботи з лічильниками у циклах.

Дійсний тип даних (float)

Тип даних float призначений для зберігання дійсних чисел, тобто чисел з дробовою частиною. У Python дробова частина відокремлюється крапкою, а не комою.

Приклади чисел типу float:

- 3.14
- 0.5
- -12.75

Приклад створення змінної типу float:

```
x = 2.5  
y = -0.75
```

Тип float застосовується у випадках, коли потрібна висока точність обчислень, наприклад:

- фінансові та економічні розрахунки;
- фізичні вимірювання;
- середні значення та статистичні дані.

Слід пам'ятати, що через особливості представлення дійсних чисел у пам'яті комп'ютера можливі незначні похибки округлення.

Рядковий тип даних (str)

Тип даних str використовується для зберігання текстової інформації. Рядок у Python береться в одинарні (' ') або подвійні (" ") лапки.

Приклади рядків:

```
name = "Python"  
city = 'Kyiv'
```

Рядки можуть містити:

- літери;
- цифри;
- пробіли;
- спеціальні символи.

Тип `str` часто застосовується для:

- введення та виведення інформації користувачем;
- збереження імен, повідомлень, текстових даних;
- обробки тексту.

Варто зазначити, що введені з клавіатури дані за допомогою функції `input()` завжди мають тип `str`, навіть якщо користувач вводить числа.

Логічний тип даних (`bool`)

Тип даних `bool` використовується для зберігання логічних значень, які можуть набувати лише двох значень:

- `True` – істина;
- `False` – хиба.

Приклад використання типу `bool`:

```
is_even = True  
is_active = False
```

Логічний тип даних застосовується для:

- перевірки умов;
- керування виконанням програм;
- роботи з операторами порівняння та умовного виконання.

Значення типу `bool` часто є результатом порівняльних операцій:

- `a = 10`
- `b = 5`
- `result = a > b # True`

Приклад:

```
a = 7
b = 3
print(a + b) # 10
```

Оператор додавання може застосовуватись також до дійсних чисел:

```
x = 2.5
y = 1.2
print(x + y) # 3.7
```

Оператор віднімання (-)

Оператор - використовується для обчислення різниці двох чисел.

Синтаксис:

```
результат = a - b
```

Приклад:

```
a = 10
b = 4
print(a - b) # 6
```

Також оператор - може використовуватися як унарний для зміни знака числа:

```
x = 5
print(-x) # -5
```

Оператор множення (*)

Оператор * використовується для множення числових значень.

Синтаксис:

```
результат = a * b
```

Приклад:

```
a = 6
b = 4
print(a * b) # 24
```

При множенні цілого числа на дійсне результат буде дійсним:

```
print(3 * 2.5) # 7.5
```

Оператор ділення (/)

Оператор / використовується для виконання ділення і завжди повертає дійсне число (float), навіть якщо результат є цілим.

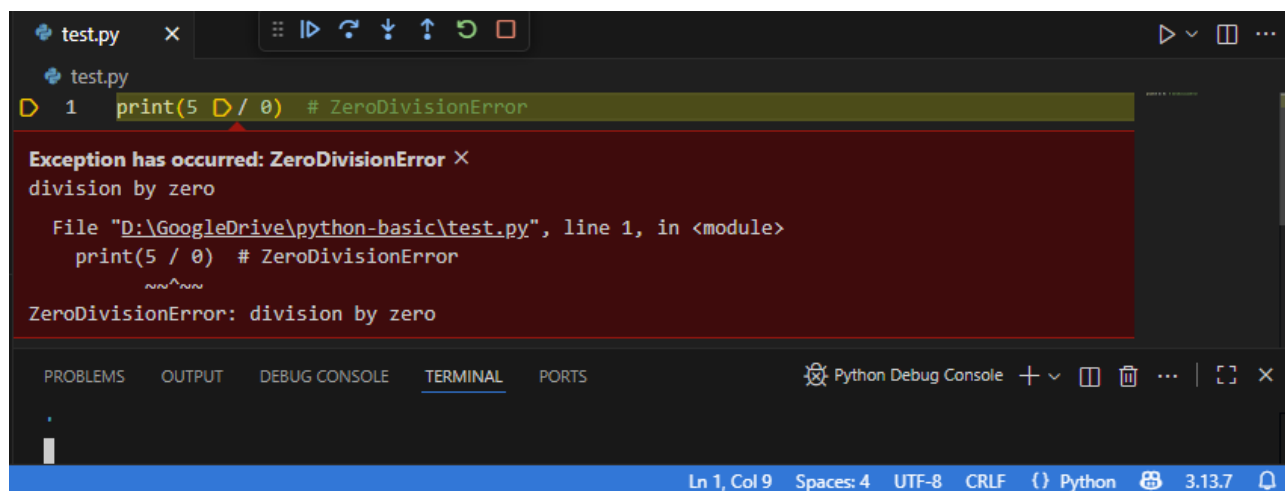
Синтаксис:

```
результат = a / b
```

Приклад:

```
print(8 / 2)    # 4.0
print(7 / 2)    # 3.5
```

При діленні на нуль виникає помилка виконання:



Оператор цілочисельного ділення (//)

Оператор // виконує цілочисельне ділення, тобто повертає цілу частину результату ділення без дробової частини.

Синтаксис:

```
результат = a // b
```

Приклад:

```
print(7 // 2)    # 3
print(9 // 2)    # 4
```

При роботі з від'ємними числами результат округлюється у менший бік:

```
print(-7 // 2)   # -4
```

Оператор знаходження залишку від ділення (%)

Оператор % використовується для обчислення залишку від ділення.

Синтаксис:

```
результат = a % b
```

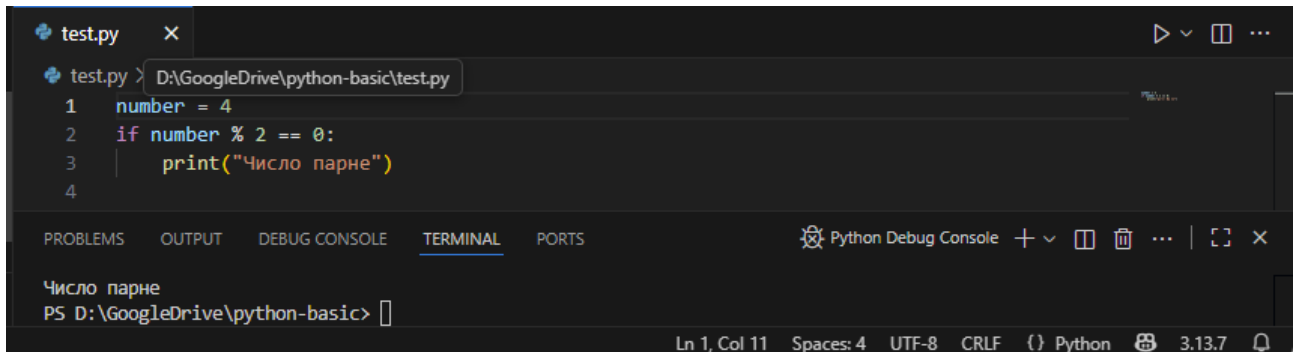
Приклад:

```
print(7 % 2)     # 1
print(10 % 3)    # 1
```

Цей оператор часто застосовується для:

- перевірки парності числа;
- циклічних обчислень;
- роботи з календарними та часовими значеннями.

Приклад перевірки парності:



```
test.py
test.py > D:\GoogleDrive\python-basic\test.py
1 number = 4
2 if number % 2 == 0:
3     print("Число парне")
4
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console
Число парне
PS D:\GoogleDrive\python-basic>
Ln 1, Col 11 Spaces: 4 UTF-8 CRLF Python 3.13.7
```

Оператор піднесення до степеня (**)

Оператор ** використовується для піднесення числа до степеня.

Синтаксис:

```
результат = a ** b
```

Приклад:

```
print(2 ** 3) # 8
print(5 ** 2) # 25
```

Можливе піднесення до дробового степеня:

```
print(9 ** 0.5) # 3.0
```

Пріоритет арифметичних операторів

Арифметичні оператори виконуються відповідно до правил пріоритету:

- 1) **
- 2) *, /, //, %
- 3) +, -

Для зміни порядку виконання використовуються круглі дужки.

Приклад:

```
result = 2 + 3 * 4 # 14
result = (2 + 3) * 4 # 20
```


Функція input()

Функція input() призначена для зчитування даних, які вводить користувач з клавіатури.

Після виконання функції програма призупиняє роботу і очікує, доки користувач введе дані та натисне клавішу Enter.

Загальний синтаксис

```
input([повідомлення])
```

Аргумент повідомлення є необов'язковим і використовується для виведення підказки користувачу.

Приклад

```
name = input("Введіть ваше ім'я: ")
```

У цьому прикладі:

- на екран виводиться текстове повідомлення;
- введене користувачем значення зберігається у змінній name.

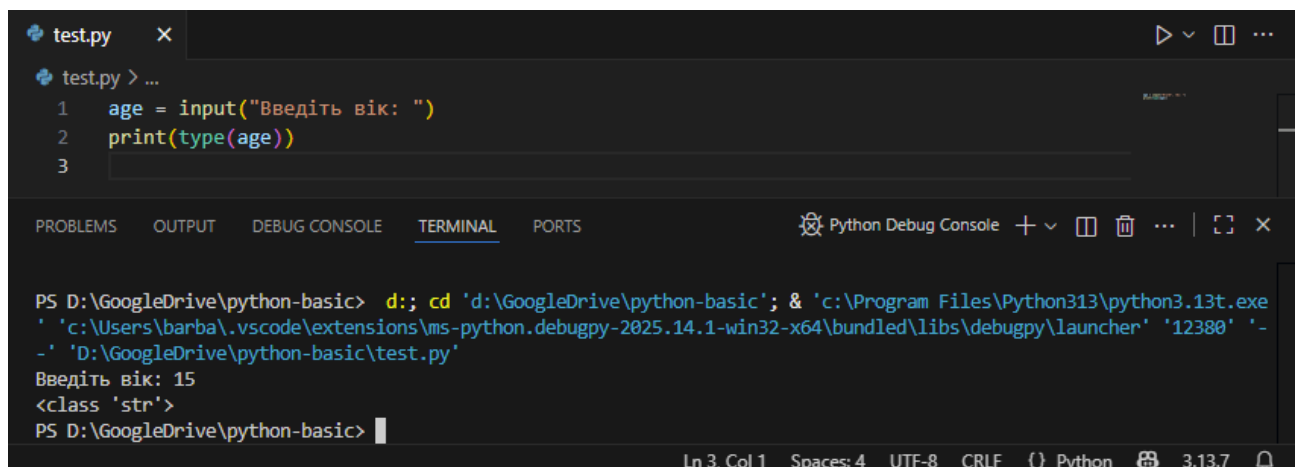
Тип даних, що повертається

Функція input() завжди повертає дані типу str (рядок), незалежно від того, що саме ввів користувач.

Приклад:

```
age = input("Введіть вік: ")
print(type(age))
```

Результат:



```
test.py x
test.py > ...
1 age = input("Введіть вік: ")
2 print(type(age))
3

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console
PS D:\GoogleDrive\python-basic> d;; cd 'd:\GoogleDrive\python-basic'; & 'c:\Program Files\Python313\python3.13t.exe' 'c:\Users\barba\.vscode\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '12380' '- -' 'D:\GoogleDrive\python-basic\test.py'
Введіть вік: 15
<class 'str'>
PS D:\GoogleDrive\python-basic>
```

Тому для виконання арифметичних операцій необхідно виконувати перетворення типів.

Перетворення введених даних

Для перетворення введених рядкових даних у числові типи використовуються стандартні функції:

- `int()` – для цілих чисел;
- `float()` – для дійсних чисел.

Приклад

```
a = int(input("Введіть перше число: "))
b = float(input("Введіть друге число: "))
```

У цьому випадку введені дані можуть бути використані в математичних обчисленнях.

Функція `print()`

Функція `print()` призначена для виведення інформації на екран.

Вона може виводити:

- текстові повідомлення;
- значення змінних;
- результати виразів;
- декілька значень одночасно.

Загальний синтаксис

```
print(значення1, значення2, ..., значенняN)
```

Приклади

```
print("Привіт, світ!")
print(a)
print("Сума:", a + b)
```

Функція автоматично додає символ переходу на новий рядок після кожного виклику.

Виведення декількох значень

Функція `print()` дозволяє виводити кілька аргументів, розділяючи їх комами.

```
x = 5
y = 10
print("x =", x, ", y =", y)
```

За замовчуванням між аргументами встановлюється пробіл.

Форматування виведення

Для зручного та зрозумілого відображення результатів застосовується форматування рядків.

Використання f-рядків:

```
result = 15
print(f"Результат обчислення: {result}")
```

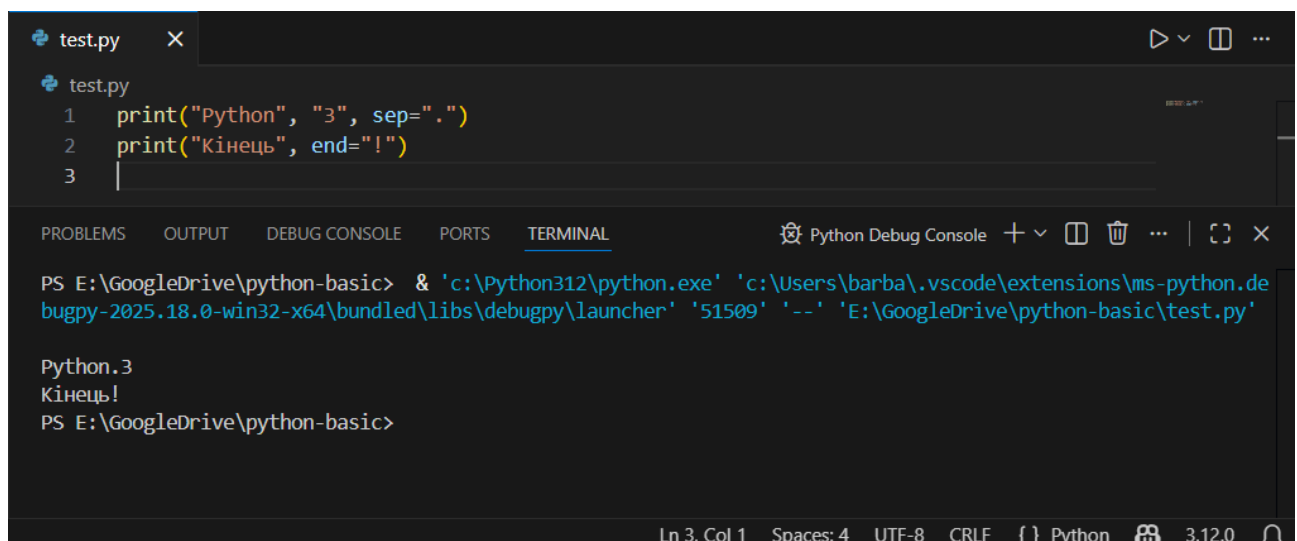
Такий спосіб є рекомендованим, оскільки підвищує читабельність коду.

Зміна параметрів виведення

Функція `print()` має додаткові параметри:

- `sep` – визначає роздільник між аргументами;
- `end` – визначає символ завершення рядка.

Приклад:



The screenshot shows a VS Code editor with a file named `test.py`. The code in the editor is:

```
1 print("Python", "3", sep=".")
2 print("Кінець", end="!")
3
```

Below the editor, the `TERMINAL` panel shows the command used to run the script and its output:

```
PS E:\GoogleDrive\python-basic> & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.de
bugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51509' '--' 'E:\GoogleDrive\python-basic\test.py'

Python.3
Кінець!
PS E:\GoogleDrive\python-basic>
```

The status bar at the bottom indicates the current position is `Ln 3, Col 1`, with `Spaces: 4`, `UTF-8` encoding, `CRLF` line endings, and the Python 3.12.0 interpreter is selected.

Типові помилки при використанні `input()` та `print()`

Під час роботи з введенням та виведенням даних найчастіше виникають такі помилки:

- використання введених даних без перетворення типу;
- спроба виконання арифметичних операцій над рядками;
- відсутність зрозумілих повідомлень для користувача.

Значення функцій input() та print() у програмуванні

Функції input() та print() є базовими інструментами взаємодії користувача з програмою. Вміння коректно використовувати ці функції є необхідною умовою для створення інтерактивних програм та подальшого вивчення більш складних тем мови Python.

1.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

1.7 Загальне завдання

Написати програму, яка:

- зчитує два числа з клавіатури;
- виконує операції додавання, віднімання, множення та ділення;
- виводить результати обчислень на екран.

1.8 Індивідуальні завдання

- 1) Програма для додавання двох чисел.
- 2) Програма для віднімання двох чисел.
- 3) Програма для множення двох введених чисел.
- 4) Програма для ділення двох чисел.
- 5) Програма для перевірки, чи є число парним.
- 6) Програма для обчислення середнього арифметичного трьох чисел.

- 7) Програма для обчислення залишку від ділення.
- 8) Програма для визначення більшого з двох чисел.
- 9) Програма для перевірки, чи є одне число дільником іншого.
- 10) Програма для конвертації температури з градусів Цельсія у Фаренгейти.

Нульовий варіант (приклад виконання)

Завдання:

- 1) Розробити програму мовою Python, яка зчитує два числа з клавіатури та обчислює:
 - суму;
 - різницю;
 - добуток;
 - частку (за умови, що друге число не дорівнює нулю).
- 2) Результати обчислень вивести на екран з пояснювальними повідомленнями.

Приклад реалізації

```
# Лабораторна робота №1
# Тема: Основи роботи з Python: змінні, типи даних та операції
# Варіант 0 (зразок виконання)

# Введення першого числа з клавіатури
a = float(input("Введіть перше число: "))

# Введення другого числа з клавіатури
b = float(input("Введіть друге число: "))

# Обчислення суми двох чисел
sum_result = a + b

# Обчислення різниці двох чисел
diff_result = a - b

# Обчислення добутку двох чисел
mult_result = a * b

# Виведення результатів обчислень
print("Сума чисел:", sum_result)
print("Різниця чисел:", diff_result)
```



```
print("Добуток чисел:", mult_result)

# Перевірка ділення на нуль
if b != 0:
    div_result = a / b
    print("Частка чисел:", div_result)
else:
    print("Ділення на нуль неможливе!")
```

Результат виконання

```
Введіть перше число: 15
Введіть друге число: 20
Сума чисел: 35.0
Різниця чисел: -5.0
Добуток чисел: 300.0
Частка чисел: 0.75
PS E:\GoogleDrive\python-basic>
```

Пояснення коду:

- для зчитування даних використовується функція ***input()***;
- оскільки ***input()*** повертає рядковий тип (***str***), введені значення перетворюються у тип ***float***;
- арифметичні операції виконуються за допомогою стандартних операторів ***+***, ***-***, *******, ***/***;
- перед виконанням ділення реалізована перевірка на ділення на нуль за допомогою умовного оператора ***if***;
- результати обчислень виводяться функцією ***print()*** з пояснювальними повідомленнями.

1.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.

- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

1.10 Контрольні питання

- 1) Які базові типи даних використовуються в Python?
- 2) Як створюються змінні в Python?
- 3) Які арифметичні оператори підтримує Python?
- 4) Чим відрізняється оператор / від //?
- 5) Як отримати залишок від ділення в Python?

1.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>